

Neuromorphic ASR

Transducer Models on Memristor Hardware

Azim Javed

`azim.javed@ml.rwth-aachen.de`

August 24, 2026

Advisor: Benedikt Hilmes and Simon Berger

Supervisor: Prof. Ralf Schlüter

Human Language Technology and Pattern Recognition
Computer Science Department, RWTH Aachen University

Outline

Introduction

Motivation

Prerequisites

Experiments

Future Work

Outline

Introduction

Motivation

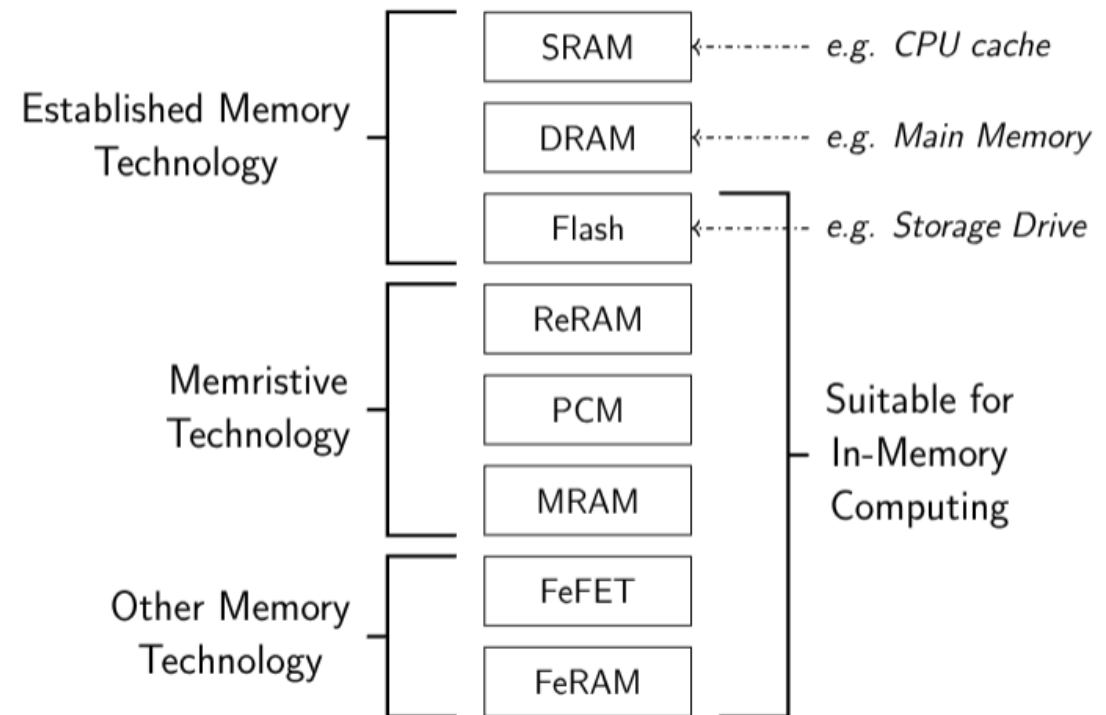
Prerequisites

Experiments

Future Work

In-Memory Computing [?]

- ▶ Various technologies exist, with varying tradeoffs:
 - ▷ Chip complexity and density
 - ▷ Energy consumption
 - ▷ Speed and reliability
- ▶ This talk focuses on **memristive** hardware, in particular **resistive RAM (ReRAM)**:
 - ▷ Key Benefits: low energy consumption, non volatility



Overview[?]

- ▶ A **memristor** is an analog computing device with a programmable resistance states.
- ▶ Enables energy-efficient implementation of neural networks.
 - ▷ Vector-matrix-multiplication (VMM) in $\mathcal{O}(1)$
- ▶ Limitations:
 - ▷ High analog inaccuracies, and thus need specific restrictions on mathematical operations.
 - ▷ Production is still in early stages, experiments are on simulated hardware [?].

Resistor:



Memristor:



Outline

Introduction

Motivation

Prerequisites

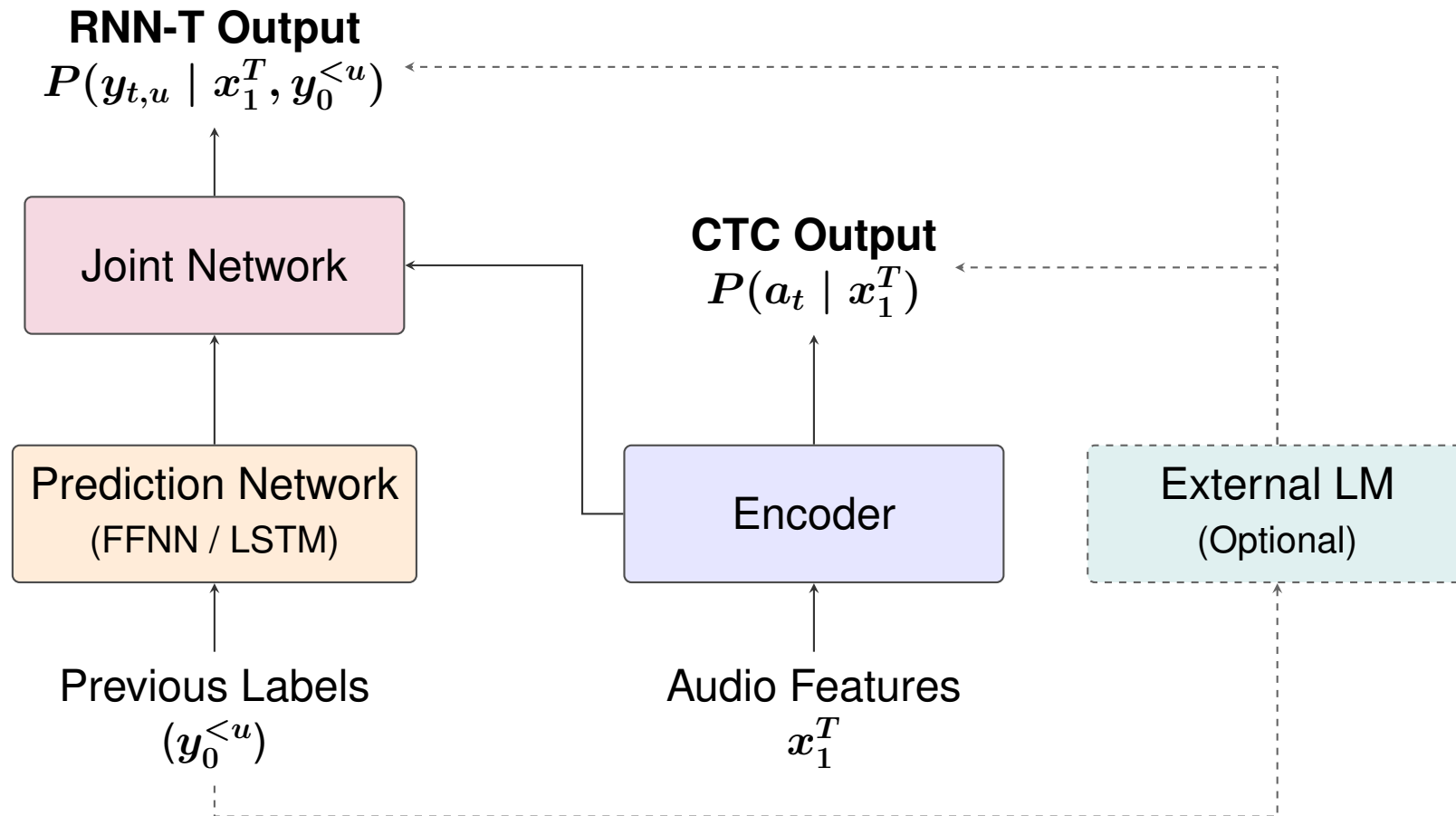
Experiments

Future Work

Motivation

- ▶ CTC-based Conformer ASR has been shown to run on (simulated) memristor hardware [Rossenbach & Hilmes⁺ 25].
- ▶ How do **transducer** models, with their added complexity compared to CTC, behave on memristor hardware?
 - ▷ Autoregressive prediction network; internal language model (ILM).
 - ▷ Joint network evaluated repeatedly per emitted label.
- ▶ The impact of deploying various components of the model to the device (encoder only vs. encoder + prediction network vs. full model).
- ▶ **Tradeoff**: more offloading $\Rightarrow$ more energy efficiency, but also more noise.
- ▶ Performance degradation across search paradigms -
 - ▷ Lexicon-free vs. tree search.
 - ▷ ILM and prior scaling - How does device noise change the optimal LM weighting?

ASR Architectures: CTC vs. Transducer



Outline

Introduction

Motivation

Prerequisites

Experiments

Future Work

Analog In Memory Computing

- ▶ Multiplication computation done via Ohm's Law:

$$I = \frac{V}{R} \quad (1)$$

- ▶ Voltage V , current I and resistance R .
- ▶ Possible to rewrite resistance as conductance G :

$$I = GV \quad \text{with} \quad G = \frac{1}{R} \quad (2)$$

- ▶ Addition done via Kirchhoff's Law for any connected node in a circuit:

$$\sum_{n=0}^N I_n = 0 \rightarrow I_0 = - \sum_{n=1}^N I_n \quad (3)$$

- ▶ Currents can be trivially added by simply connecting electric lines.

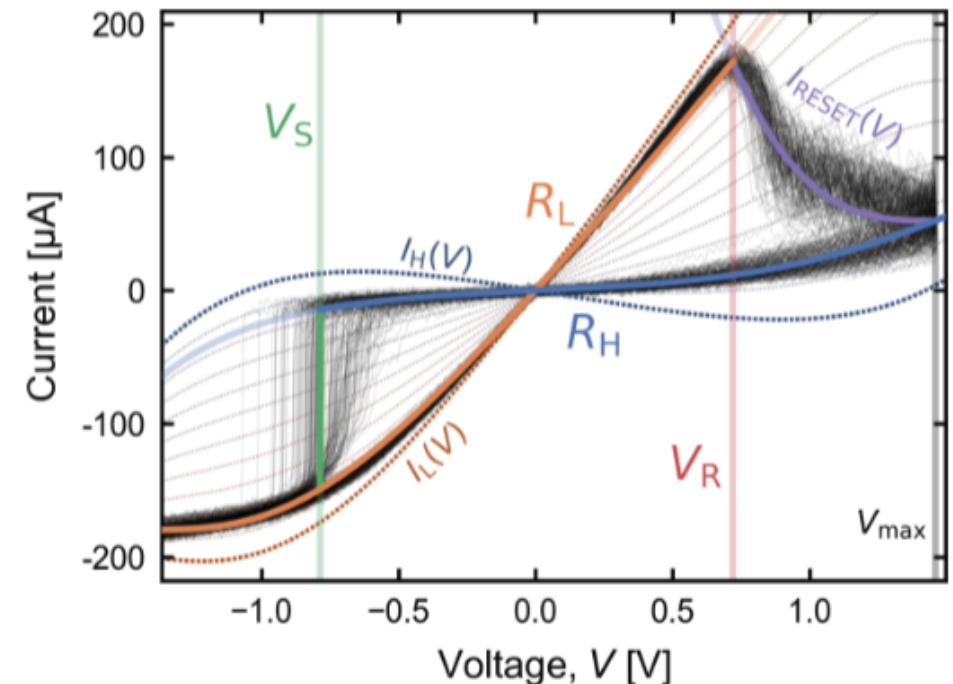
The “Memristor”

Program weights by setting the resistance:

- ▶ Apply $-V > -V_S \rightarrow$ set to low resistance
- ▶ Apply $V > V_R \rightarrow$ set to high resistance
- ▶ Intermediate states possible
- ▶ State is physically stored (non-volatile memory)

Problems:

- ▶ Resulting resistance state varies with each programming cycle
- ▶ Non-linear relation of current and voltage

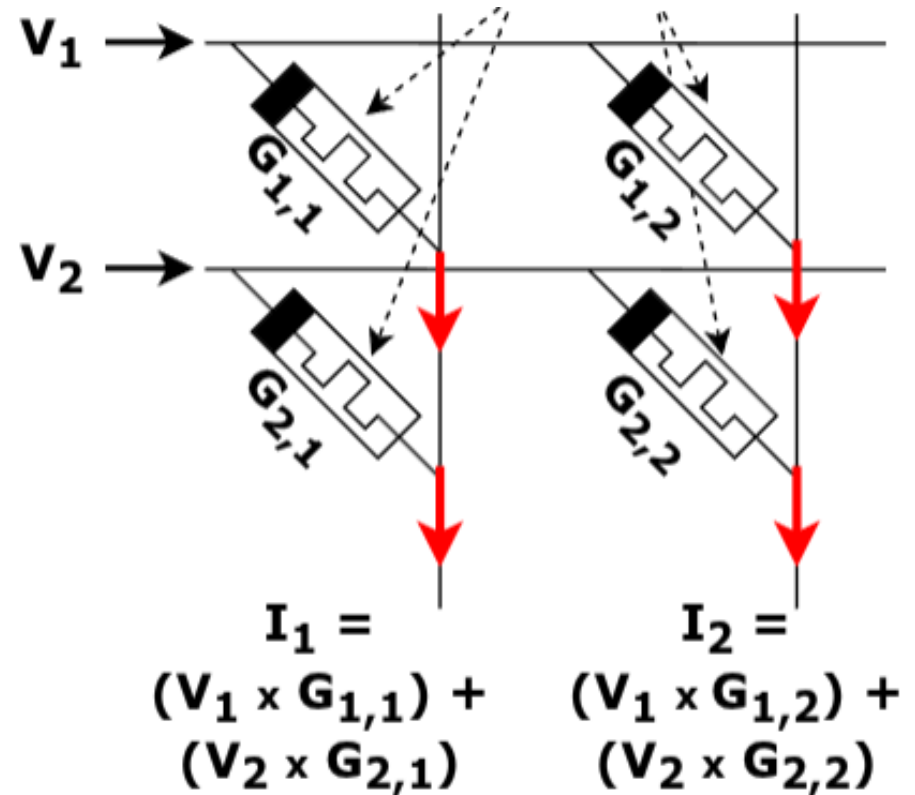


Memristor Array (1)

- ▶ Memristors as connecting element between vertical and horizontal circuit lines
- ▶ Each memristor is a weight of a matrix
- ▶ Column sum via simple line connection
- ▶ Weight matrices are typically fixed during application

Problems:

- ▶ Only positive, non-zero weights possible
- ▶ Current leakage prohibits a true zero



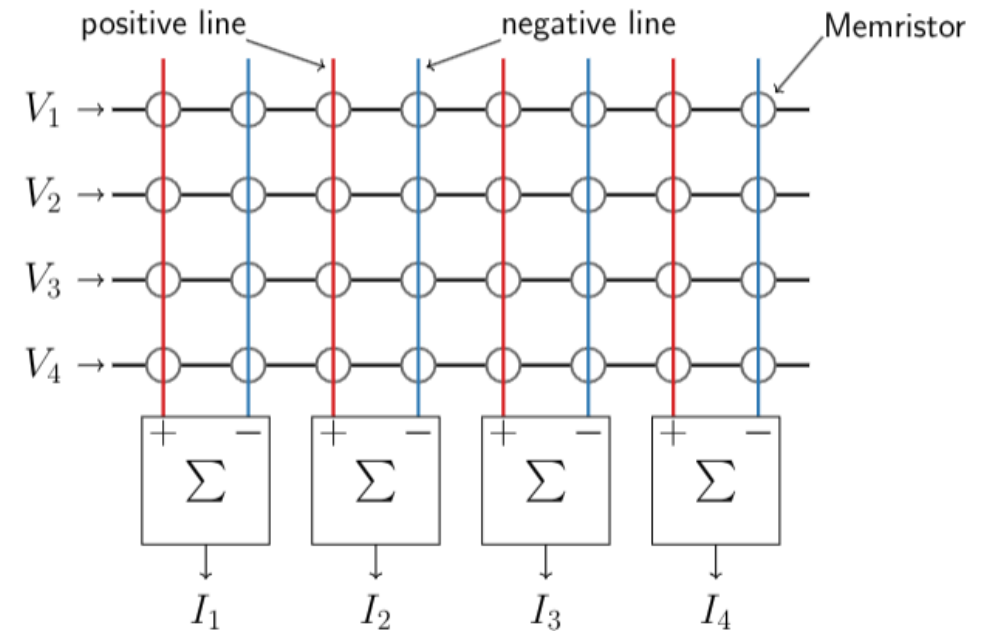
Memristor Array (2)

- ▶ Use positive and negative bitlines to create memristor pairs
- ▶ Each row-wise pair of memristors represents a weight
- Differential pair allows for negative and zero weight values:

$$I_j = \sum_{i=1}^N V_i G_{i,j}^+ - \sum_{i=1}^N V_i G_{i,j}^-$$

Problems:

- ▶ Cannot set paired memristors to identical conductances
- Results close to zero, but not guaranteed



From Single Memristor to Layer Module: Concepts

► Multiple concepts to implement mapping:

▷ **Tiling** of weight matrix:

- ▷ Arbitrary dimensions by tiling into smaller fixed-size arrays
- ▷ E.g., 512×512 matrix into 16 crossbar arrays of size 128×128

▷ **Slicing** of weight values to memristor states:

- ▷ Currently binary setting (high vs. low resistance)
- ▷ Stacking of memristors for each magnitude bit (k -bit weight requires $k - 1$ arrays)

▷ **Pairing** of positive (+) and negative (−) bitlines:

- ▷ Differential pairs represent signed matrix entries

From Single Memristor to Layer Module: Mapping

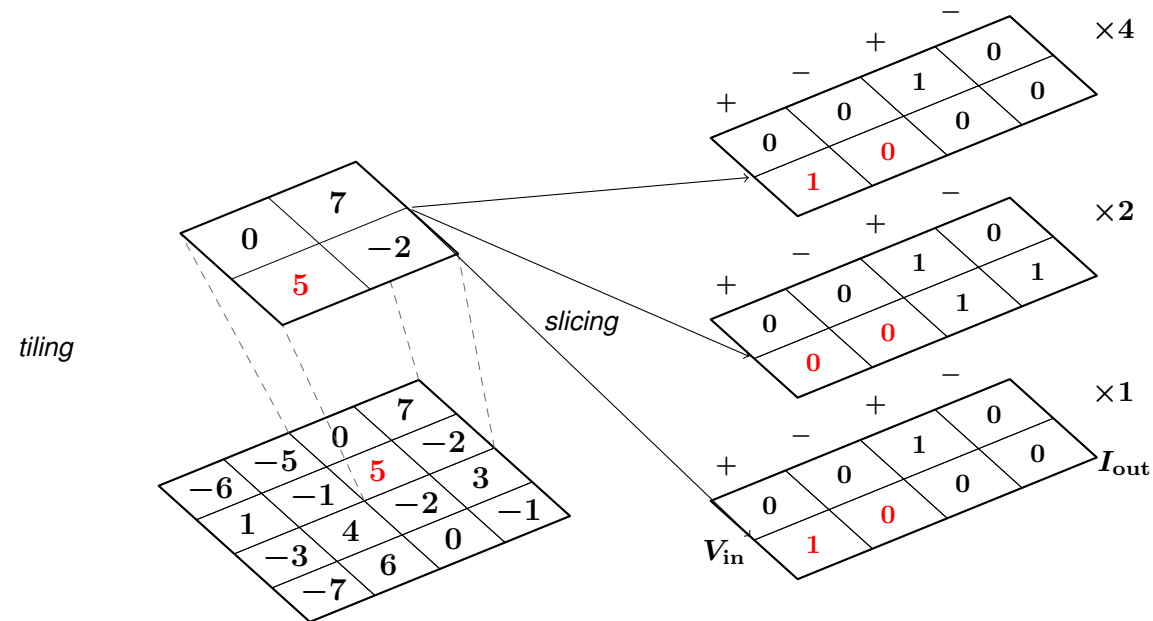


Figure: Exemplary mapping of a 4-bit quantized weight onto 3 memristor arrays.

Non-Idealities and Synaptogen

- ▶ Ideal crossbars are assumed by many simulators; real devices exhibit:
 - ▷ Device-to-device and cycle-to-cycle programming variability
 - ▷ Thermal (Johnson–Nyquist) noise
 - ▷ Finite DAC/ADC resolution
 - ▷ Imprecise intermediate (non-binary) conductance levels
- ▶ Synaptogen [Hennen & Brackmann⁺ 24]: generative device model based on a vector autoregression, fitted to 3M+ measured programming cycles of 130 nm memristors (STMicroelectronics).
- ▶ In our experiments (PyTorch, SynaptogenML): linear layers are replaced by crossbar simulation, the rest of the network stays conventional.

Quantization-Aware Training

- ▶ Memristor execution effectively means: low-precision (stacked binary) weights + 8-bit DAC/ADC on activations.
- ▶ Post-training quantization (PTQ) collapses at low precision; quantization-aware training (QAT) [Gholami & Kim⁺ 21] inserts fake quantization into the training from epoch 1.
 - ▷ Symmetric per-tensor quantization, zero-point fixed to 0 (required by the pairwise encoding).

Table: WER (%) on TED-LIUMv2 dev, Conformer CTC, 4-gram LM [Rossenbach & Hilmes⁺ 25].
Memristor: mean $\pm$ std over 10 differently programmed devices.

Weight bits	FP32	8	6	5	4	3	2
PTQ	7.2	7.2	7.9	8.9	11.4	30.7	–
QAT		7.3	7.7	7.4	7.8	8.3	22.1
Memristor (simulated)		8.3	8.3	8.3	8.9	9.2	–

- ▶ Up to 3-bit weights: $\leq 25\%$ relative WER degradation on device (9.2 vs. 7.2).

Outline

Introduction

Motivation

Prerequisites

Experiments

Future Work

Experimental Setup

- ▶ Data: LibriSpeech [Panayotov & Chen⁺ 15] (960 h), BPE subword targets (128 merges → 184 labels).
- ▶ Features: 80 log-mel filters, 10 ms frame shift; SpecAugment during training.
- ▶ Encoder (shared by all models): 12-layer Conformer [Gulati & Qin⁺ 20], 512-dim.
- ▶ Model types:
 1. **CTC** [Graves & Fernández⁺ 06]: encoder only.
 2. **FFNN-transducer**: prediction network is a 2-layer feed-forward net (640-dim, tanh) over the previous-label embedding.
 3. **Full-context transducer** [Graves 12]: LSTM prediction network (standard RNN-T).
- ▶ Quantization: QAT, per-tensor symmetric, e.g. 4-bit weights / 8-bit activations; zero-point 0.
- ▶ Stages: (1) FP32 baseline → (2) QAT (fake quantization) → (3) memristor execution (Synaptogen crossbar simulation, multiple re-programmings per run).

Experiment Matrix

- ▶ Full matrix: 3 (model types) $\times$ 5 (search types) $\times$ 3 (stages).
- ▶ Search types:
 1. Lexicon-free, no LM
 2. Lexicon-free + BPE LSTM LM
 3. Prefix-tree beam search, no LM
 4. Prefix-tree beam search + 4-gram LM
 5. Prefix-tree beam search + BPE LSTM LM
(+ large-beam tree variants, beam 1024/2048)
- ▶ Prior/ILM correction and external LM scales are swept at decoding time.
- ▶ Reporting:
 - ▷ Primary tables: one per model type (search types $\times$ stages).
 - ▷ Appendix: tables across models at the same stage, for easy comparison.
- ▶ Expectation: device deployment changes the interplay between prior/ILM correction and the LM — quantization and noise reshape the acoustic scores, so the optimal LM weighting should shift.

Outline

Introduction

Motivation

Prerequisites

Experiments

Future Work

Future Work

- ▶ Error analysis: which error types does device noise introduce or amplify?
- ▶ Countermeasures: denoising (e.g. MLP), search parameters, LM parameters, softmax temperature.
- ▶ Fine-tuning of the (quantized) model; effect of noise introduction during training.
- ▶ Generalization: comparison across different datasets / data conditions.

Summary

- ▶ Extend memristor-based ASR [Rossenbach & Hilmes⁺ 25] from CTC to transducer models.
- ▶ Full matrix on LibriSpeech: 3 model types $\times$ 5 search types $\times$ 3 stages (baseline / QAT / memristor).
- ▶ Key questions: component offloading (encoder vs. encoder + prediction), real noise impact, lexfree vs. tree search behavior, prior/ILM–LM interplay.

Thank you for your attention!

Reference

- 📄 A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, K. Keutzer.
A Survey of Quantization Methods for Efficient Neural Network Inference, Aug. 2021.
[arXiv:2103.13630.](#)
- 📄 A. Graves, S. Fernández, F. Gomez, J. Schmidhuber.
Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks.
[In *Proc. ICML*, pp. 369–376, Pittsburgh, PA, June 2006.](#)
- 📄 A. Graves.
Sequence Transduction with Recurrent Neural Networks.
[In *Proc. ICML*, Edinburgh, Scotland, June 2012.](#)
[Workshop on Representation Learning, arXiv:1211.3711.](#)
- 📄 A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, R. Pang.
Conformer: Convolution-Augmented Transformer for Speech Recognition.
[In *Proc. Interspeech*, pp. 5036–5040, Shanghai, China, Oct. 2020.](#)

- 📄 T. Hennen, L. Brackmann, T. Ziegler, S. Siegel, S. Menzel, R. Waser, D. J. Wouters, D. Bedau.
Synaptogen: A Cross-Domain Generative Device Model for Large-Scale Neuromorphic Circuit Design.
IEEE Transactions on Electron Devices, Vol. 71, No. 9, pp. 5345–5353, Sept. 2024.
[arXiv:2404.06344](#).
- 📄 V. Panayotov, G. Chen, D. Povey, S. Khudanpur.
Librispeech: an ASR Corpus Based on Public Domain Audio Books.
In Proc. IEEE ICASSP, pp. 5206–5210, Queensland, Australia, April 2015.
- 📄 N. Rossenbach, B. Hilmes, L. Brackmann, M. Gunz, R. Schlüter.
Running Conventional Automatic Speech Recognition on Memristor Hardware: A Simulated Approach.
In Proc. Interspeech, Rotterdam, The Netherlands, Aug. 2025.
[arXiv:2505.24721](#).

Appendix: Device VMM Operation Statistics

operation	average	std-dev	min	max
1.0×1	1.016	0.063	0.707	1.27
0.1×1	$0.989 \cdot 10^{-1}$	$0.062 \cdot 10^{-1}$	$0.688 \cdot 10^{-1}$	$1.24 \cdot 10^{-1}$
0.01×1	$0.985 \cdot 10^{-2}$	$0.069 \cdot 10^{-2}$	$0.721 \cdot 10^{-2}$	$1.20 \cdot 10^{-2}$
1.0×0	$8.52 \cdot 10^{-4}$	$4.75 \cdot 10^{-2}$	-0.2751	0.2455
1.0×0.5	0.476	0.094	0.109	1.06

Table: Hardware measurement statistics for simulated memristor VMM operations [Rossenbach & Hilmes⁺ 25].